

Knuth-Morris-Pratt Algorithm

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

1/1

Basic definitions

An **alphabet** is a non-empty, finite set of symbols, called **letters**, for example a, b, c etc.

A **word** is a finite series of letters from an alphabet, like abracadabra. A letter is a word.

Just as letters can be joined to make up words, so can words: the word $u \cdot v$ is made of the letters of word u followed by the letters of word v , for instance, if $u = \text{back}$ and $v = \text{up}$, then $u \cdot v = \text{backup}$. This operation is called **concatenation**.

Concatenation is associative: $(u \cdot v) \cdot w = u \cdot (v \cdot w)$. The operator may be omitted: $(uv)w = u(vw)$. Concatenation is a non-commutative product, so it has a neutral element ε , called the **empty word**: $u \cdot \varepsilon = \varepsilon \cdot u = u$.

2/1

Basic definitions

We denote the **repetition** of a word with an exponent, e.g. $a^3 = aaa$. The **length** of a string x is the length of the associated sequence and is noted $|x|$. So $|\varepsilon| = 0$, $|a| = 1$ and $|a^3| = 3$.

A word x is a **factor** of a word y if there exists two words u and v (possibly empty) such that $y = uxv$. In other words, u occurs in y .

The word x is a **prefix** of y , noted $x \leq y$, if $u = \varepsilon$, that is, if $y = xv$. Moreover, it is a **proper prefix**, noted $x < y$, if $v \neq \varepsilon$.

Given $y = uxv$, the word x is a **suffix** of y if $v = \varepsilon$. Furthermore, it is a **proper suffix** if $u \neq \varepsilon$.

Let a range over letters, and x and y over words. The prefix relation is easy to define inductively:

- $\forall y. \varepsilon \leq y$;
- $\forall a, x, y. x \leq y \Rightarrow a \cdot x \leq a \cdot y$.

3/1

Basic definitions

A letter in a word can be uniquely characterised by a natural number, called **index**, assuming that the first letter has index 0. Read

<https://www.cs.utexas.edu/~EWD/transcriptions/EWD08xx/EWD831.html>

For example, if $x = \text{top}$, then the letter at index 0 is written $x[0] = \text{t}$ and the one at index 2 is $x[2] = \text{p}$.

A factor x of y can be identified by the index of $x[0]$ in y .

The end of the factor can also be given; for example, $x = \text{sit}$ is a factor of $y = \text{curiosity}$ at index 5, written $y[5, 7] = x$. This is sometimes called a **slice** in some programming languages.

Sometimes the upper bound can be left unspecified: $y[i \dots]$, which is a short-cut for $y[i, |y| - 1]$, if $i < |y|$. That is, $y[i \dots]$ is a **suffix** of y . Likewise: $y[\dots i]$ is a **prefix** of y .

4/1

The prefix predicate

An imperative definition:

```
PREFIX( $x, t$ )  
1   $i \leftarrow 0$   
2  while  $i < \min(|x|, |t|)$  do  
3     $i \leftarrow 1 +$  if  $x[i] = t[i]$  then  $i$  else  $|x|$   
4  PREFIX  $\leftarrow i = |x|$ 
```

Does it terminate? What test cases would be meaningful? Is it correct? Is it optimal?

5/1

The prefix predicate

A single static assignment definition:

```
PREFIX( $x, t$ )  
1  PREFIX  $\leftarrow$  PREFIX'( $0, x, t$ )  
  
PREFIX'( $i, x, t$ )  
1  if  $i < \min(|x|, |t|)$   
2    then PREFIX'  $\leftarrow$  PREFIX'( $1 +$  if  $x[i] = t[i]$  then  $i$  else  $|x|, x, t$ )  
3    else PREFIX'  $\leftarrow i = |x|$ 
```

6/1

Naïve search

Factor matching is common in text editing, although it is usually better known as **exact string matching** in the academic field of *text algorithmics* (sometimes called *stringology*).

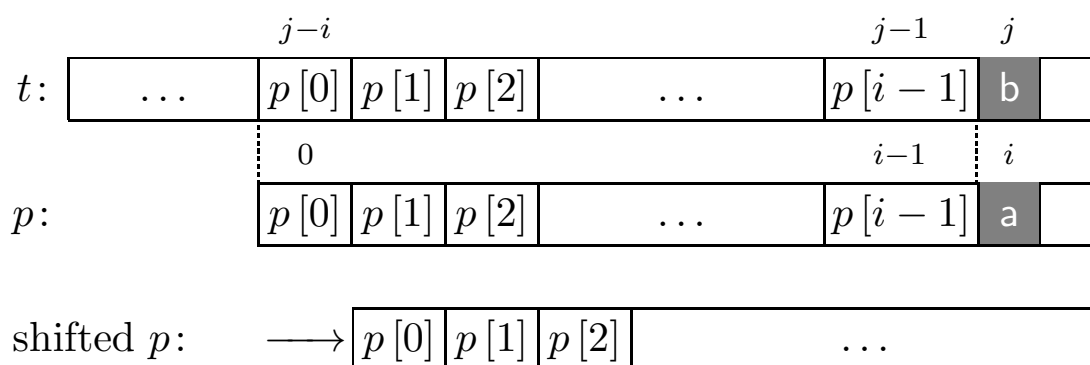
Because of the asymmetric nature of the search, the word p to find will from now on be called the **pattern**, and the word t the **text**.

Assuming p and t are not empty, the naïve search simply consists in checking whether p is a prefix of t : if so, we are done, otherwise, the pattern is shifted by one letter to the right and, again, it is checked whether it is a prefix of that text suffix.

If p cannot be shifted anymore because its end would surpass the end of t , then it is not a factor: the search failed.

7/1

Naïve search



Naïvely matching pattern p against text t (failure in grey), where $p[i] \neq t[j]$ (the letters **a** and **b** are not relevant in themselves).

8/1

Naïve search

An imperative program:

```
    NAIVE( $p, t$ )
1   $i, j \leftarrow 0, 0$ 
2  while  $i < |p|$  and  $j < |t|$  do
3      if  $p[i] = t[j]$ 
4          then  $i, j \leftarrow i + 1, j + 1$ 
5          else  $i, j \leftarrow 0, j - i + 1$        $\triangleright$  We slide  $p$  by 1.
6   $\text{NAIVE} \leftarrow i = |p|$ 
```

What test cases would be meaningful?

Would it be clearer to use two loops?

How about a version with single static assignments and slices? Without slices?

9/1

Naïve search

The input is a pattern p of length m and a text t of length n

What configuration of the input leads to the minimal number of comparisons? What is that number?

In the worst case, p is not in t and the test at line 3 always fails on the last letter of p , leading to make p slide of one position at line 5 the latest as possible.

An example of positive match in the worst case is $p = a^{m-1}b$ and $t = a^{n-1}b$, with $m \leq n$.

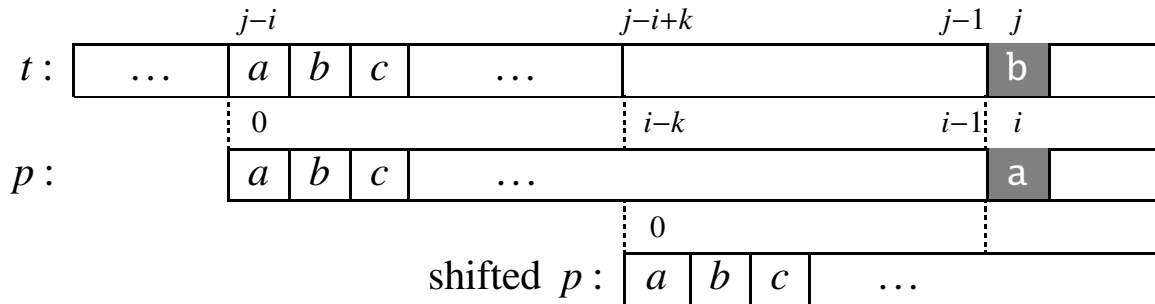
There are $n - m + 1$ positions in t to compare with the first letter of p , and since there are m letters in p , it makes a total of $(n - m + 1)m = nm - m^2 + m < |t| \times |p|$ positions.

What about the worst case when there is no match?

10/1

Morris-Pratt algorithm

In case of mismatch, the naïve algorithm resumes comparing the first letters of p without using the information of the partial success, to wit, we know $p[0, i-1] = t[j-i, j-1]$ and $p[i] \neq t[j]$.



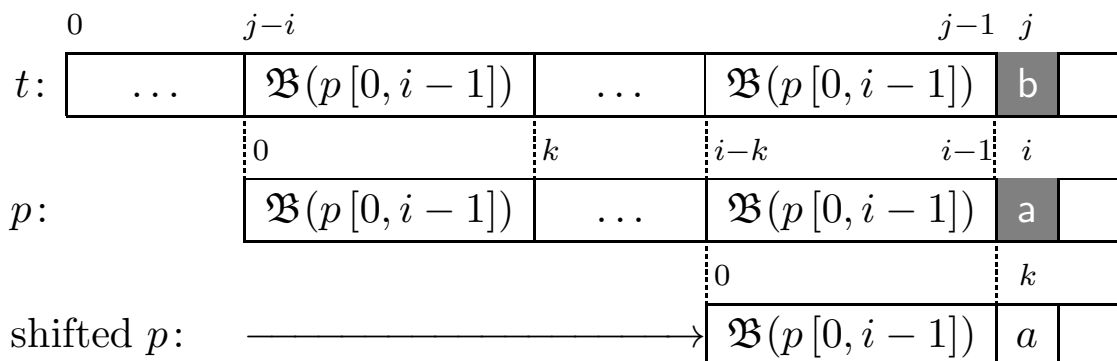
If a new search starts in t at position $j-i+k$, with $1 \leq k \leq i$, it compares a prefix of p to $t[j-i+k \dots]$. Since $t[j-i+k \dots j-1] = p[i-k, i-1]$, it compares a prefix of p to a suffix of $p[\dots i-1]$, that is, to a part of itself!

11/1

Morris-Pratt algorithm

The important point here is that these comparisons are *independent* of the text t , since they apply to part of the pattern p itself.

Contrary to naïve factoring, letters in the text are compared in a strictly increasing order, which enables **searches in data streams**.



where $\mathfrak{B}(p)$ is the **maximum border** of p : the longest prefix of p that is also a suffix of $p[\dots i-1]$.

12/1

Morris-Pratt algorithm

The strategy devised here is to **compute the maximum borders of all prefixes of p before we start the search** in the text, and then use this information to avoid repeating the same comparisons at different positions in the text.

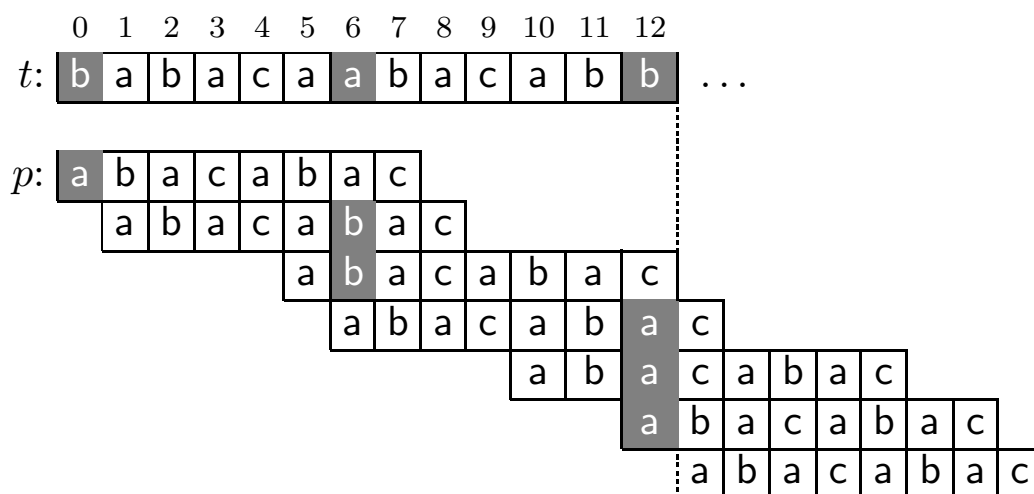
This **preprocessing** on p will indeed allow to recognise the sole configurations where the search may succeed.

In other words, we need to understand borders.

13/1

Morris-Pratt algorithm

But before that, let us consider an example where, in the end, p is not found to be a factor of t . As usual, letters on a grey background correspond to mismatches. It is clear that $\mathcal{B}(a) = \varepsilon$, for all letters a .



(The text is not shown in its entirety.)

14/1

Morris-Pratt algorithm

The **border** of a non-empty word y is a proper prefix of y which is also a suffix.

To understand the case of the **empty border**, consider $\mathcal{B}(\text{abac}) = \varepsilon$, simply because we have $\text{abac} = \underline{\varepsilon}\text{abac}\underline{\varepsilon}$.

For example, the word ‘abacaba’ has three borders: ε , ‘a’ and ‘aba’. The last one is the **maximum border** and we write $\mathcal{B}(\underline{\text{abacaba}}) = \text{aba}$.

Note that **borders can overlap**; for example, $\mathcal{B}(\underline{\text{aaaa}}) = \mathcal{B}(\text{aaaa}) = \text{aaa}$.

The border above must be a proper prefix, so it cannot equal ‘aaaa’.

15/1

Morris-Pratt algorithm

As often when working at the level of specifications, we seek an inductive definition for the function at hand, in this instance, the maximum border of a word.

We then have the choice of finding either $\mathcal{B}(ay)$ or $\mathcal{B}(ya)$, with $y \neq \varepsilon$, assuming that we know $\mathcal{B}(y)$.

Since we are interested in knowing the maximum borders of all the *prefixes* of a given pattern, the ya is more suitable ($y < ya$).

The idea is to recursively consider $\mathcal{B}(y) \cdot a$:

- if it is a prefix of y , then $\mathcal{B}(ya) = \mathcal{B}(y) \cdot a$;
- otherwise, we consider the border of $\mathcal{B}(y) \cdot a$.

16/1

Morris-Pratt algorithm

Consider the following examples where y and $\mathfrak{B}(y)$ are given:

$$\begin{aligned} y = \text{abaabb}, \quad \mathfrak{B}(y) &= \varepsilon, \quad \mathfrak{B}(y \cdot b) = \mathfrak{B}(\mathfrak{B}(y) \cdot b) = \mathfrak{B}(b) = \varepsilon; \\ y = \text{baaaba}, \quad \mathfrak{B}(y) &= \text{ba}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(y) \cdot a = \text{baa}; \\ y = \text{abbbab}, \quad \mathfrak{B}(y) &= \text{ab}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}^2(y) \cdot a = a. \end{aligned}$$

For example, $\mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^2(y) \cdot a) = \mathfrak{B}^3(y) \cdot a$:

$$\begin{aligned} y \cdot a &= \begin{array}{|c|c|c|c|} \hline \mathfrak{B}(y) & b & \dots & \mathfrak{B}(y) & a \\ \hline \end{array} \\ \mathfrak{B}(y) \cdot a &= \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^2(y) & c & \dots & \mathfrak{B}^2(y) & a \\ \hline \end{array} \\ \mathfrak{B}^2(y) \cdot a &= \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^3(y) & a & \dots & \mathfrak{B}^3(y) & a \\ \hline \end{array} \end{aligned}$$

17/1

Morris-Pratt algorithm

Formally, for all words $y \neq \varepsilon$ and any letter a ,

$$\mathfrak{B}(a) := \varepsilon; \quad \mathfrak{B}(y \cdot a) := \begin{cases} \mathfrak{B}(y) \cdot a, & \text{if } \mathfrak{B}(y) \cdot a \leq y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \text{otherwise.} \end{cases}$$

Unfolding the equations yields:

$$\begin{aligned} \mathfrak{B}(ya) &= \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \mathfrak{B}(y) \cdot a &\not\leq y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a) &= \mathfrak{B}(\mathfrak{B}^2(y) \cdot a), & \mathfrak{B}^2(y) \cdot a &\not\leq \mathfrak{B}(y); \\ &\vdots & \vdots \\ \mathfrak{B}(\mathfrak{B}^{p-2}(y) \cdot a) &= \mathfrak{B}(\mathfrak{B}^{p-1}(y) \cdot a), & \mathfrak{B}^{p-1}(y) \cdot a &\not\leq \mathfrak{B}^{p-2}(y); \end{aligned}$$

and $\varepsilon \notin \{y, \mathfrak{B}(y), \dots, \mathfrak{B}^{p-2}(y)\}$, so $\mathfrak{B}(ya) = \mathfrak{B}(\mathfrak{B}^{p-1}(y) \cdot a)$.

If $\mathfrak{B}^{p-1}(y) = \varepsilon$, then $\mathfrak{B}(ya) = \mathfrak{B}(a) = \varepsilon$, else the unfolding resumes until we find the smallest $q > p$ such that $\mathfrak{B}(\mathfrak{B}^{q-1}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^q(y) \cdot a)$ with $\mathfrak{B}^q(y) \cdot a \leq \mathfrak{B}^{q-1}(y)$.

18/1

Morris-Pratt algorithm

Because a border is a proper prefix, that is to say, $\mathfrak{B}(y) < y$, we have

$$\mathfrak{B}^2(y) = \mathfrak{B}(\mathfrak{B}(y)) < \mathfrak{B}(y),$$

so $\mathfrak{B}^q(y) \cdot a \leq \mathfrak{B}^{q-1}(y) < \dots < \mathfrak{B}(y) < y$. Therefore $\mathfrak{B}^q(y) \cdot a \leq y$, since $q > 0$, and $\mathfrak{B}(ya) = \mathfrak{B}^q(y) \cdot a$.

Gathering all the cases, an equivalent definition of the maximum border:

$$\mathfrak{B}(ya) = \begin{cases} \mathfrak{B}^q(y) \cdot a, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ \varepsilon, & \text{otherwise;} \end{cases}$$

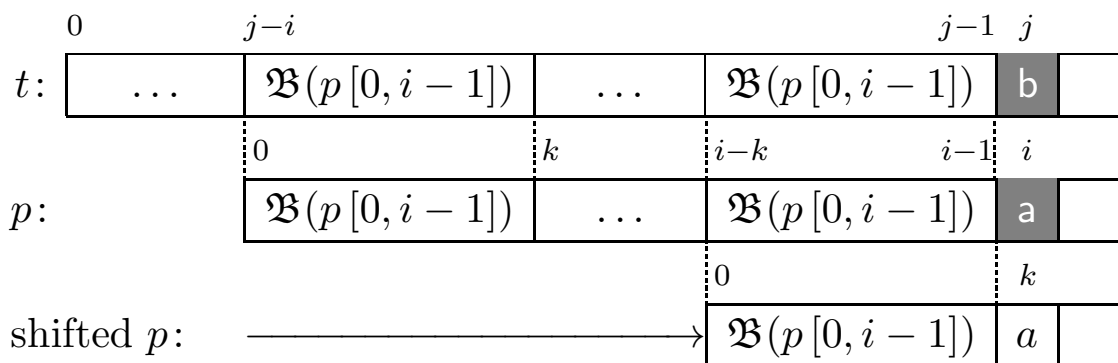
with the additional constraint that q **must be as small as possible** (so we define the maximum border).

This form of the definition of $\mathfrak{B}$ is simpler because it does not contain an embedded call.

19/1

Morris-Pratt algorithm

We do not actually need to compute the maximum borders of all the prefixes of the pattern. Let us have a look back at



What we need is the offset corresponding to the **length of the maximum border** being considered (k , in the figure).

20/1

Morris-Pratt algorithm

Let us define a function $\mathcal{N}_x$ on all the prefixes of a word x as follows:

$$\mathcal{N}_x(|y|) := |\mathfrak{B}(y)|, \text{ for all } x \text{ and } y \neq \varepsilon \text{ such that } y \leq x.$$

For reasons which will be clear soon, this function is called the **failure function** of x . An equivalent definition is

$$\mathcal{N}_x(i+1) = |\mathfrak{B}(x[\dots i])|, \text{ for all } x \text{ and } i \text{ such that } 0 \leq i < |x|.$$

Let's take the equations defining the maximum borders and take the lengths on both sides:

$$|\mathfrak{B}(ya)| = \begin{cases} |\mathfrak{B}^q(y) \cdot a| = 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ |\varepsilon| = 0, & \text{otherwise.} \end{cases}$$

21/1

Morris-Pratt algorithm

If $ya \leq x$, then $|\mathfrak{B}(ya)| = \mathcal{N}_x(|ya|) = \mathcal{N}_x(|y| + 1)$. Let $i := |y| > 0$.

$$\mathcal{N}_x(i+1) = \begin{cases} 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

Let's set aside for a moment $\mathfrak{B}^q(y) \cdot a \leq y$ and return to the definition of $\mathcal{N}$, from which we deduce:

$$\mathcal{N}_x^q(|y|) = |\mathfrak{B}^q(y)|, \text{ with } y \leq x,$$

which we can prove by complete induction on q . Let us call this property $P(q)$. Trivially, we have $P(0)$. Let us suppose $P(n)$ for all $n \leq q$: this is the induction hypothesis. We suppose $y \leq x$ and prove now $P(q+1)$:

$$\mathcal{N}_x^{q+1}(|y|) = \mathcal{N}_x^q(\mathcal{N}_x(|y|)) = \mathcal{N}_x^q(|\mathfrak{B}(y)|) \doteq |\mathfrak{B}^q(\mathfrak{B}(y))| = |\mathfrak{B}^{q+1}(y)|,$$

where $(\doteq)$ is a valid application of the induction hypothesis because we have $\mathfrak{B}(y) < y \leq x$. This proves $P(q+1)$ and the induction principle entails that $P(n)$ holds for all $n \geq 0$.

22/1

Morris-Pratt algorithm

So

$$\mathcal{N}_x(i+1) = \begin{cases} 1 + \mathcal{N}_x^q(i), & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

There is one part of the definition that we did not use: $\mathfrak{B}(a) := \varepsilon$. It implies

$$\mathcal{N}_x(1) = \mathcal{N}_x(|a|) = |\mathfrak{B}(a)| = |\varepsilon| = 0$$

and, since the definition of $\mathcal{N}$ implies $\mathcal{N}_x(1) = 1 + \mathcal{N}_x(0)$, so $\mathcal{N}_x(0) = -1$.

We can now return to the last missing bit:

$$\mathfrak{B}^q(y) \cdot a \leq y \text{ and } ya \leq x \text{ and } |y| = i.$$

Since $\mathfrak{B}^q(y) < y$, those are equivalent to the following:

$$y[|\mathfrak{B}^q(y)|] = a \Leftrightarrow y[\mathcal{N}_x^q(i)] = a \Leftrightarrow x[\mathcal{N}_x^q(i)] = x[|y|] \Leftrightarrow x[\mathcal{N}_x^q(i)] = x[i].$$

23/1

Morris-Pratt algorithm

Gathering everything:

$$\mathcal{N}_x(0) = -1 \quad \text{and} \quad \mathcal{N}_x(i+1) = \begin{cases} 1 + \mathcal{N}_x^q(i), & \text{if } x[\mathcal{N}_x^q(i)] = x[i]; \\ 0, & \text{otherwise;} \end{cases}$$

where q is the smallest nonzero natural satisfying the condition.

This can be further simplified into

$$\mathcal{N}_x(0) = -1 \quad \text{and} \quad \mathcal{N}_x(i+1) = 1 + \mathcal{N}_x^q(i), \text{ with } i \geq 0,$$

and $q > 0$ is the smallest natural such that $\mathcal{N}_x^q(i) = -1$ or $x[\mathcal{N}_x^q(i)] = x[i]$.

24/1

Morris-Pratt algorithm

```

NEXT(x, i)                                     ▷  $\mathcal{N}_x(i)$  with  $0 \leq i < |x|$ 
1  if  $i = 0$ 
2    then NEXT  $\leftarrow -1$                      ▷  $\mathcal{N}_x(0) = -1$ 
3    else  $j \leftarrow i - 1$ 
4      repeat                                   ▷  $q > 0$  times
5         $j \leftarrow \text{NEXT}(x, j)$ 
6      until  $j = -1$  or  $x[j] = x[i - 1]$          ▷ Sequentially
7      NEXT  $\leftarrow 1 + j$                      ▷  $\mathcal{N}_x(i) = 1 + \mathcal{N}_x^q(i - 1)$ 

```

The table of the maximum borders for the proper prefixes of the word ‘abacabac’:

$x =$	a	b	a	c	a	b	a	c
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	0	1	0	1	2	3

25/1

Morris-Pratt algorithm

Static single-assignment form, assuming $0 \leq i < |x|$:

```

NEXT(x, i)
1  NEXT  $\leftarrow$  if  $i = 0$  then -1 else  $1 + \text{FIX}(x, \text{NEXT}(x, i, i - 1))$ 

FIX(x, i, j)
1  FIX  $\leftarrow$  if  $j = -1$  or  $x[j] = x[i - 1]$  then  $j$  else  $\text{FIX}(x, i, \text{NEXT}(x, j))$ 

```

26/1

Morris-Pratt algorithm

The problem here is inefficiency:

1. successive calls with the same arguments trigger again the same computations,
2. the recursive calls are costly.

It is much preferable to have $\mathcal{N}$ as an array than as a function:

1. the values of $\mathcal{N}$ are computed only once each
2. and are accessed in constant time instead of relying on function calls.

27/1

Morris-Pratt algorithm

The idea is to compute all the values of $\mathcal{N}_x(i)$ for *increasing* values of i :

```

NEXT(x)
1  next[0] ← -1
2  for i ← 1 to |x| - 1 do
3      j ← i - 1
4      repeat
5          j ← next[j]                                ▷  $\mathcal{N}_x[j] < j < i$ 
6      until j = -1 or x[j] = x[i - 1]
7      next[i] ← 1 + j                                ▷  $\mathcal{N}_x(i) = 1 + \mathcal{N}_x^q(i - 1)$ 
8  NEXT ← next
```

28/1

Morris-Pratt algorithm

To find out the best configuration of a pattern p of length m that yields the minimum number of letter comparisons, let us assume that the inner loop in the definition of NEXT is always executed exactly once:

$$\begin{array}{ll} \mathcal{N}_x(m) = 1 + \mathcal{N}_x(m-1) & x[\mathcal{N}_x(m-1)] = x[m-1] \\ \mathcal{N}_x(m-1) = 1 + \mathcal{N}_x(m-2) & x[\mathcal{N}_x(m-2)] = x[m-2] \\ \vdots & \vdots \\ \mathcal{N}_x(2) = 1 + \mathcal{N}_x(1) & x[\mathcal{N}_x(1)] = x[1] \\ \mathcal{N}_x(1) = 1 + \mathcal{N}_x(0) & \\ \mathcal{N}_x(0) = -1 & \end{array}$$

So $\mathcal{N}_x(j) = j - 1$ and $x[j-1] = x[j]$, for $j > 0$, so the number of letter comparisons for p is $m - 1$. An example of pattern is $p = a^m$.

29/1

Morris-Pratt algorithm

To find the worst configuration of a pattern p of length m that yields the maximum number of letter comparisons, let us assume that the inner loop in the definition of NEXT is executed as many times as possible, that is to say, the prefix test $x[j-1] = x[j]$ fails until $j = -1$.

In other words, $\mathcal{N}_x(j)$ must decrease as slowly as possible, which is actually the best case: $\mathcal{N}_x(j) = j - 1$.

In other words, the worst case is the best case followed by a letter that is not in the best case prefix: $p = a^{m-1}b$, where $a \neq b$. We know that the number of comparisons (equalities) for a^{m-1} is $m - 2$.

We need to add the comparisons (inequalities) of b to all the a : $x[j] \neq x[m-1]$ for $0 \leq j < m-1$, that is, $m - 1$.

In total, the number of comparisons in the worst case is $(m - 2) + (m - 1) = 2m - 3$.

30/1

Morris-Pratt algorithm

Let us parameterise the search with the failure function:

```

    FIND( $p, t, next$ )
1   $i, j \leftarrow 0, 0$ 
2  while  $i < |p|$  and  $j < |t|$  do
3      if  $i = -1$  or  $p[i] = t[j]$ 
4          then  $i, j \leftarrow i + 1, j + 1$ 
5          else  $i \leftarrow next[i]$ 
6  FIND  $\leftarrow$  if  $i = |p|$  then  $j - |p|$  else -1

```

$\triangleright j$ invariant

So the Morris-Pratt algorithm is

```

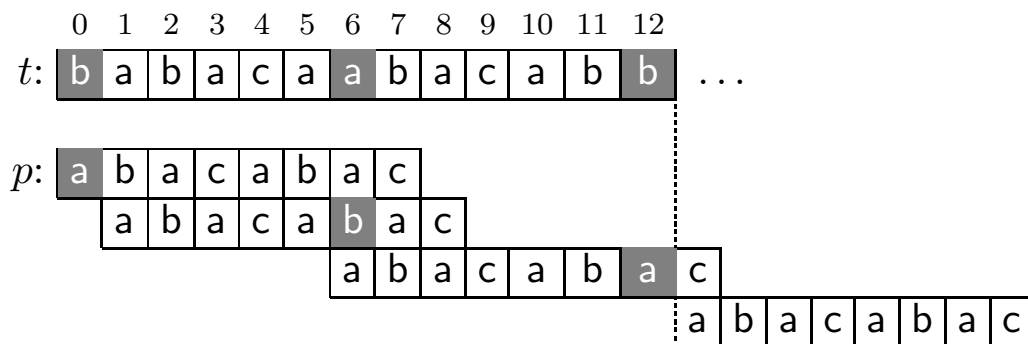
    MP( $p, t$ )
1  MP  $\leftarrow$  FIND( $p, t, NEXT(p)$ )

```

31/1

Knuth-Morris-Pratt algorithm

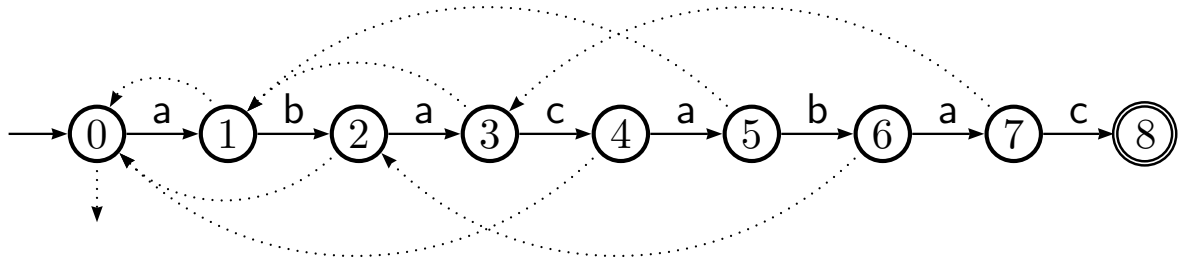
Knuth proposed an improvement: if sliding the pattern entails comparing the same letter from the pattern with the same letter in the text, **we should slide again** because the same mismatch will happen, and again until we find a new letter.



32/1

Knuth-Morris-Pratt algorithm

Let us see the need for this improvement in another way, by representing graphically $\mathcal{N}_x$, with $x = \text{abacabac}$, as we know it:



This is the Morris-Pratt *deterministic finite automaton* (DFA) for the word *abacabac*. The table we gave earlier was:

$x =$	a	b	a	c	a	b	a	c
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	0	1	0	1	2	3

33/1

Knuth-Morris-Pratt algorithm

We will here only describe informally such automata.

Consider that the circles, called **states**, contain the values of i from the table. The edges, called **transitions**, between two states are of two kinds: either solid and carrying a letter, called **label**, or dotted and going backwards. The sequence of states throughout solid edges make the word $x = \text{abacabac}$.

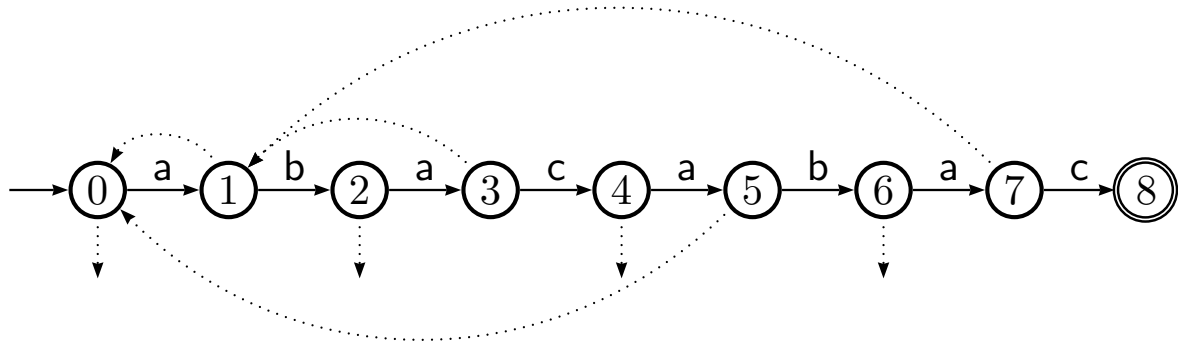
The rightmost state is distinguished by a double circling because it marks the end of x . There is a back edge between state i and j only if $\mathcal{N}_x(i) = j$.

The leftmost state has an incoming, solid edge without a source and a dotted, outgoing edge. The former simply denotes the beginning of the word and the latter corresponds to the special value $\mathcal{N}_x(0) = -1$.

34/1

Knuth-Morris-Pratt algorithm

In terms of the automaton, the improvement consists in following the back edges



Intuitively, it goes as follows. At a given state with the outgoing label a , follow the back edge to another state. If the outgoing label is a then repeat, otherwise we are done and we process another state. If the back edge arrives at state 0 and the outgoing label is still a , it loses its destination.

35/1

Knuth-Morris-Pratt algorithm

In terms of a table, this DFA is equivalent to

$x =$	a	b	a	c	a	b	a	c
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	-1	1	-1	0	-1	1

$\text{NEXT}'(x)$

- 1 $next \leftarrow \text{NEXT}(x)$
- 2 **for** $i \leftarrow 1$ **to** $|x| - 1$ **do**
- 3 $j \leftarrow next[i]$
- 4 **if** $j \geq 0$ **and** $x[j] = x[i]$ **then** $next[i] \leftarrow next[j]$
- 5 $\text{NEXT}' \leftarrow next$

36/1

Knuth-Morris-Pratt algorithm

We can fuse the two loops into one. The idea is to maintain j as the failure state of i before optimisation (that is, original edges $i \dashrightarrow j$).

```

    NEXT'(x)
1  next[0], j ← -1, 0
2  for i ← 1 to |x| - 1 do
3      if x[j] = x[i]
4      then next[i] ← next[j]
5      else next[i] ← j
6          while j ≥ 0 and x[j] ≠ x[i] do j ← next[j]
7      j ← j + 1
8  NEXT' ← next
```